Ecommerce Website — Product Requirements Document (PRD)

Version: 1.0

Status: Draft for Product/Engineering review

Target production launch: 31 March 2027

Primary stack: Next.js, TypeScript, Node.js/Express microservices, Redux Toolkit, Zod, PostgreSQL, Prisma, AWS

Planning baseline: 2-week sprints beginning 26 October 2026; reduced capacity during 21 December–1 January; production launch window 29–31 March 2027.

> **Planning note:** This is a PM-ready baseline with explicit assumptions. Product/legal/operations must confirm market, currency, payment provider, shipping/tax rules, return policy, review eligibility, forecast traffic, and available team capacity in Sprint 1. These are launch dependencies, not details to leave implicit.

1. Executive summary

Build a secure, responsive, production-ready single-store ecommerce platform that lets customers discover and evaluate products, maintain a cart/wishlist, check out, pay through a hosted/tokenized provider flow, track orders, and request eligible cancellations/returns. Store operators need a role-gated admin experience to manage catalog, pricing promotions, inventory, orders, fulfilment, and customer support workflows. The system must be observable, testable, accessible, SEO-capable, and deployable on AWS by **31 March 2027**.

The launch plan prioritizes reliable end-to-end purchase and operational correctness over optional marketplace or personalization features. The initial release assumes **one merchant/storefront**. Multi-vendor seller onboarding, seller payouts, advanced recommendations, loyalty, subscriptions, and native mobile apps are outside launch scope unless the product owner explicitly reprioritizes them.

2. Product vision and outcomes

2.1 Vision

Provide customers with a trustworthy, low-friction shopping experience while enabling staff to operate the store safely without direct database access.

2.2 Goals

- Customers can browse/search, understand product/variant availability and pricing, and complete a purchase.
- Orders and payments reconcile correctly, including retries, duplicate webhooks, declines, cancellation and refund paths.
- Staff can maintain products, stock, promotions, orders and fulfilment through a permissioned admin interface.
- The product meets agreed launch quality gates for security, accessibility, performance, SEO, reliability and support readiness.
- Development is incremental, testable, documented, and deployable through repeatable CI/CD.

2.3 Success metrics (targets to validate in Sprint 1)

Metric	Launch target / measurement
Purchase journey completion	At least 95% pass rate across automated P0 end-to-end scenarios in release candidate testing
Payment/order consistency	100% of sampled test payments reconcile to a valid order state; no duplicate charge/order on retry tests
Critical defects	Zero open Sev-1/Sev-2 defects at go/no-go
Performance	Core public pages target LCP <= 2.5s at p75 on agreed representative device/network; key read APIs target p95 <= 500ms under agreed test load, excluding third-party latency
Availability	Initial monthly service availability objective: 99.5%, subject to agreed operating budget and architecture
Accessibility	Core journeys reviewed against WCAG 2.2 AA; no blocker-level keyboard or checkout issues at launch
Security	No unresolved critical/high security findings without documented risk acceptance
Operations	Successful backup restore rehearsal, alert routing, deployment and rollback rehearsal before launch
Analytics	Product view, search, add-to-cart, checkout start, purchase, payment failure and refund events validated without exposing sensitive payment data

Targets must be confirmed against budget, traffic profile, provider limitations, and team capability. They are not a substitute for testing.

3. Personas and access model

Persona / role	Primary needs	Allowed capabilities
Guest shopper	Discover products and build a cart before creating an account	Browse/search/product detail, guest cart, begin checkout; account required if business rules demand it
Registered customer	Purchase and manage own shopping/order activity	Profile, addresses, cart, wishlist, checkout, own orders, eligible cancellation/return, review subject to policy
Catalog manager	Maintain product content and merchandising	Product/category/variant/media management, publication status, approved bulk import/export
Inventory/fulfilment operator	Keep stock and orders accurate	Inventory adjustment with reason, order lookup, fulfilment/tracking status transitions
Customer support	Help customers with order issues	Restricted customer/order lookup, notes or requests allowed by policy; no unnecessary payment/credential access
Administrator	Configure store and grant roles	Role/user administration, policy/configuration, full operational access subject to audit and least privilege

Finance/operations (optional distinct role)	Reconcile payments/refunds	Read payment status, reconciliation exports, initiate approved refunds if delegated

Authorization rule: Hide unauthorized UI actions for clarity, but enforce every permission on the server. Default deny. Admin actions that affect price, stock, payment/refund, order status, customer data, or permissions must be audited.

4. Scope and release boundaries

4.1 P0 — required for launch

- Responsive storefront and reusable accessible UI components.
- Registration, verification, login/logout, password reset, session refresh/revocation and RBAC.
- Category navigation, product listing/detail, variants/SKUs, images, search, sorting and core filters.
- Catalog administration, publication status, stock visibility and authorized inventory adjustment.
- Guest/customer cart, login merge, server-authoritative price calculation, coupons/promotions.
- Checkout address, shipping option, tax calculation approach, order summary and stock reservation.
- Payment provider sandbox-to-production integration using hosted/tokenized card handling; signed webhooks; idempotency and reconciliation.
- Order creation, history/detail, status timeline, confirmation emails and invoice/receipt baseline.
- Admin order search, fulfilment state updates, coupon management, audit trail and restricted support lookup.
- Approved cancellation/refund/return request workflow.
- Required policy/help pages, SEO technical baseline, analytics, monitoring, backups, CI/CD, regression/security/performance/accessibility testing.

4.2 P1 — include only after P0 path is safe

- Product ratings/reviews and moderation if policy and moderation ownership are approved.
- Bulk CSV product import/export with row-level errors.
- Search typo tolerance or a dedicated search engine if native Postgres search is insufficient.
- Curated related products, expanded admin reporting, support notes, advanced email preferences.

4.3 Explicitly out of scope for 31 March 2027 unless reapproved

- Multi-vendor marketplace, seller registration, seller payouts and seller-specific storefronts.
- Native iOS/Android applications.
- Loyalty/points, subscriptions, gift cards, complex bundles, affiliate/referral system.
- AI personalization or recommendation engine beyond simple related products.
- International multi-currency/multi-tax jurisdictions beyond the approved launch market.
- Warehouse management system, carrier aggregation, or complex ERP integration beyond chosen launch integrations.
- Fully automated fraud scoring; use payment provider capabilities and documented manual review rules if needed.

5. Key customer and operator journeys

5.1 Browse to purchase

1. Customer opens a category or uses search.
2. Filters/sort update results and URL state; empty, loading and error states are clear.
3. Customer opens a product, inspects images/details, selects a variant and quantity.
4. Server validates that the item is published, purchasable and available, then adds it to cart.
5. Cart shows current prices and applicable promotion feedback.
6. Customer selects/enters delivery address and shipping option.
7. Server recalculates items, discounts, shipping and tax; any stale price/stock requires confirmation.
8. Customer proceeds to the provider-hosted/tokenized payment flow.
9. Signed provider event confirms payment; order transitions to the correct state; stock commits and confirmation is sent.
10. Customer sees order history/detail and later fulfilment updates.

5.2 Failed or interrupted checkout

- Declined/cancelled payment does not mark the order paid.
- Retry reuses an idempotency key or creates a safe new attempt according to provider semantics; no duplicate charge/order.
- Pending payments expire according to configured timeout; stock reservations release safely.
- Webhook event duplication/out-of-order arrival is handled idempotently and through reconciliation.
- Customer sees clear next steps without provider secrets or internal errors.

5.3 Store operations

10. Authorized catalog manager creates/updates a product, variant, SKU, media and status.
11. Inventory operator adjusts stock with a reason; system records actor, time, SKU, previous and new quantity.
12. Staff search order, inspect payment/fulfilment timeline, and apply only valid status transitions.
13. Refund/cancellation follows policy and is performed through the provider adapter; order status is reconciled from provider response/events.
14. Audit logs support investigation; routine tasks do not require direct SQL edits.

6. Functional requirements

Priorities: **P0** must ship; **P1** launch if capacity permits after all P0 gates pass; **P2** post-launch.

6.1 Platform, API and service foundations

ID	Requirement	Priority	Acceptance criteria
PLAT-01	Services expose health/readiness checks and structured logs	P0	Health includes dependency readiness where appropriate; no secrets/PII in logs; requests carry correlation IDs
PLAT-02	APIs use a consistent versioning, pagination, error and response convention	P0	Validation and business errors have stable codes; internal stack traces are not sent to clients
PLAT-03	All untrusted inputs are	P0	Invalid body/query/path values

	validated using Zod at API boundaries		return safe 4xx responses; validation is covered by tests
PLAT-04	Cross-service calls have timeouts, bounded retries and documented failure modes	P0	A failing downstream service does not cause unbounded requests or duplicate writes
PLAT-05	Service configuration is environment-based and validated at startup	P0	Missing required configuration fails fast; secrets come from approved secret management
PLAT-06	Database schema changes use reviewed, repeatable migrations	P0	CI tests migration from clean DB; rollback/forward-fix policy documented

6.2 Identity, account and authorization

ID	Requirement	Priority	Acceptance criteria
ID-01	Customer can register with validated identity fields	P0	Duplicate email handled safely; password is never stored or logged in plaintext
ID-02	Customer verifies email using expiring single-use token	P0	Expired/reused token rejected; unverified-account policy is enforced
ID-03	Customer can log in and log out	P0	Generic failure message prevents account enumeration; session revocation works
ID-04	Refresh credentials rotate and are revoked safely	P0	Reuse detection/revocation strategy documented and tested; refresh secrets stored hashed if persisted
ID-05	Customer can request password reset	P0	One-time expiring reset flow; rate limits; response does not reveal account existence
ID-06	Customer can view/update own profile and addresses	P0	Ownership enforced server-side; field validation and audit rules applied
ID-07	Admin routes and actions enforce RBAC	P0	Automated tests prove customers cannot call admin APIs; permission checks are server-side
ID-08	Authentication and sensitive flows have rate limits and abuse protections	P0	Thresholds configurable; lockout/challenge behavior does not allow trivial denial of service

Session guidance: Prefer secure, HttpOnly, SameSite cookies for refresh/session material when compatible with the chosen architecture; protect cookie-authenticated state-changing requests against CSRF. Keep short-lived access credentials out of persistent browser storage where feasible. Exact design requires an ADR and security review.

6.3 Catalog, product and media

ID	Requirement	Priority	Acceptance criteria

CAT-01	Staff can manage categories and category hierarchy	P0	Slugs are unique; inactive categories do not appear to customers; invalid hierarchy is rejected
CAT-02	Staff can create/edit/archive/publish products	P0	Required fields and valid price enforced; draft/archived products are not purchasable
CAT-03	Product supports variants/SKUs and variant attributes	P0	SKU unique; variant-specific price/stock is respected; invalid combinations cannot be ordered
CAT-04	Customer can view paginated product listings	P0	Stable ordering, category context and pagination; handles empty/error/loading states
CAT-05	Customer can view product detail, images and attributes	P0	Selected variant drives SKU/availability/price; inaccessible media handled gracefully
CAT-06	Product media uses authorized upload and controlled delivery	P0	Type/size validation; no public write access; uploads associated with product/variant
CAT-07	Product listing supports keyword search, sort and core filters	P0	Price/availability/category filters compose correctly; URL captures shareable state
CAT-08	Public product/category pages have search-engine metadata	P0	Unique title/description, canonical URL, sitemap inclusion and appropriate structured data
CAT-09	Related products and advanced search relevance	P1	Module fails gracefully; search query set reviewed by product owner

6.4 Cart, wishlist and pricing

ID	Requirement	Priority	Acceptance criteria
CART-01	Guest can add, update and remove items	P0	Cart persists according to approved session strategy; server checks status, variant and stock
CART-02	Customer can merge guest cart after login	P0	Duplicate lines merge deterministically; removed/unavailable items are reported clearly
CART-03	Server is the authority for monetary totals	P0	Client-submitted prices are ignored; money uses integer minor units/decimal-safe rules
CART-04	Cart displays current subtotal, discount and availability	P0	Repriced/changed items are identified before order submission
CART-05	Coupon eligibility and use limits are validated server-side	P0	Active window, minimum spend, product/category scope, limits and stacking policy enforced atomically
CART-06	Customer can maintain a	P1	Wishlist private to owner;

	wishlist		unavailable items clearly labeled; move-to-cart revalidates stock

6.5 Checkout, shipping and tax

ID	Requirement	Priority	Acceptance criteria
CHK-01	Customer can select or add checkout address	P0	Address ownership and required fields validated; selected address stored as order snapshot
CHK-02	Customer sees valid shipping options and costs	P0	Eligibility and costs are derived from approved launch rules/provider; impossible delivery is blocked
CHK-03	System calculates tax using approved adapter/rules	P0	Inputs and result are traceable; changes require re-confirmation; rules approved for launch market
CHK-04	Checkout presents an authoritative order review	P0	Item, discounts, shipping, tax and total reconcile; server recalculates before payment
CHK-05	Stock is reserved during pending payment and released safely	P0	Reservation timeout works; simultaneous buyers cannot oversell beyond policy
CHK-06	Checkout creation is idempotent	P0	Duplicate submit/retry does not create duplicate order/charge; idempotency scope is defined
CHK-07	Customer sees clear recovery for declined/cancelled/expired payment	P0	No false paid state; cart/order can be safely retried according to policy

6.6 Payments, orders and fulfilment

ID	Requirement	Priority	Acceptance criteria
PAY-01	Payment integration uses hosted/tokenized provider flow	P0	Card data is not stored or logged by platform; provider selected and production account approved
PAY-02	Provider webhooks are signature-verified and idempotent	P0	Invalid signatures rejected; duplicate/replayed events do not duplicate effects
PAY-03	Payment/order states follow an explicit state machine	P0	Invalid transitions blocked; pending/paid/failed/cancelled/refunded states reconcile to provider evidence
PAY-04	Payment and order reconciliation is observable	P0	Mismatches are logged/alerted; staff have a documented reconciliation procedure
ORD-01	System creates an order with immutable item/price/address snapshots	P0	Later catalog/address changes do not rewrite historical order values

ORD-02	Customer can view own order list/detail and status timeline	P0	Ownership enforced; timeline reflects only valid state transitions
ORD-03	Staff can search orders and update fulfilment state	P0	Role-restricted; allowed transitions only; actor/time are audited
ORD-04	Customer receives transactional order notifications	P0	Email retries safely; duplicate provider events do not send duplicate messages
ORD-05	Invoice/receipt is available with approved fields	P0	Values match final order/payment snapshot; jurisdictional requirements approved by PM/finance
ORD-06	Shipping/tracking identifier is saved and exposed where supported	P0	Status and tracking link are validated; customer sees updated fulfilment timeline

6.7 Admin, promotions and support

ID	Requirement	Priority	Acceptance criteria
ADM-01	Admin landing page provides operational summary	P0	Counts and alerts respect role permissions and do not expose unnecessary PII
ADM-02	Catalog staff manage products/categories/variants/media	P0	Required fields and permission checks enforced; publication status controls visibility
ADM-03	Inventory staff adjust stock with reason	P0	Non-negative stock policy enforced; every adjustment has actor/time/reason and before/after values
ADM-04	Staff manage coupons and promotion rules	P0	Backend validation matches admin UI; use limits cannot be bypassed via API
ADM-05	Operations staff update order fulfilment	P0	Invalid transitions blocked; actions are audited and customer notifications are triggered where configured
ADM-06	Sensitive admin actions are auditable/searchable	P0	Audit record includes actor, action, entity, timestamp and correlation ID; excludes secrets
ADM-07	Support has least-privilege customer/order lookup	P1	Support can locate cases without access to credentials or full payment instrument data
ADM-08	Bulk product CSV import/export has row-level validation	P1	Invalid rows reported; import preview and authorization required; changes logged

6.8 Cancellation, returns, refunds and reviews

ID	Requirement	Priority	Acceptance criteria

RET-01	Cancellation eligibility depends on approved order state/policy	P0	Customer sees eligibility; state, stock and payment side effects are consistent and audited
RET-02	Customer can request a return/refund for eligible items	P0	Reason/item/quantity captured; eligibility validated; status visible to staff/customer
RET-03	Refund calls use provider adapter and reconcile asynchronously	P0	Refund never exceeds captured amount; duplicate refund is prevented; provider status reconciles with order
REV-01	Customer can submit a product rating/review under approved eligibility rules	P1	Moderation status and abuse control work; rejected/hidden reviews do not affect public aggregation

6.9 Notifications, content and analytics

ID	Requirement	Priority	Acceptance criteria
MSG-01	Verification/reset/order/ shipment/cancel/refund emails use approved templates	P0	Responsive rendering; links expire as designed; no secrets in content/logs
MSG-02	Transactional messages are separated from marketing consent	P0	Required order emails are not suppressed by marketing opt-out; consent is recorded where applicable
CNT-01	Policy/help/contact pages are available	P0	Privacy, terms, shipping, refund/cancellation and contact content approved before launch
ANA-01	Core ecommerce funnel events are captured	P1	Event names and payloads documented; no card data, credentials or unnecessary PII sent
ANA-02	Basic funnel and operations dashboards are verified	P1	Event counts checked in staging; ownership and retention decisions recorded

7. Business rules and state models

7.1 Money and pricing

- Represent money in integer minor units or a clearly defined decimal-safe money type; never use binary floating-point arithmetic for final monetary calculations.
- The server is authoritative for item price, discount eligibility, shipping, tax, and total. The browser may display estimates but cannot submit trusted totals.
- The order stores a snapshot of product name/SKU, quantity, unit price, discount allocation, tax, shipping, currency, address and policy-relevant values at purchase time.
- Coupon stacking is disabled by default unless the product owner explicitly approves rules. Usage-limit checks must be concurrency-safe.

7.2 Inventory and concurrency

- Inventory tracks on-hand, reserved and sellable quantities, or an equivalent auditable model.
- Stock is checked when adding to cart and revalidated during checkout; cart stock is not a guarantee.
- Reservation expiry releases stock after abandoned/failed payment. Reservation jobs must be idempotent.
- Inventory adjustments require a reason and audit event. Prevent negative sellable stock unless the business explicitly adopts backorders/preorders.

7.3 Order/payment state (baseline; finalize with provider)

- **Order:** PENDING_PAYMENT -> CONFIRMED/PAID -> PROCESSING -> SHIPPED -> DELIVERED with policy-controlled branches to CANCELLED, RETURN_REQUESTED, RETURNED, and REFUNDED.
- **Payment:** CREATED -> REQUIRES_ACTION/PENDING -> SUCCEEDED or FAILED/CANCELLED; refund lifecycle is separately tracked (REFUND_PENDING -> REFUNDED/REFUND_FAILED).
- Payment provider evidence is authoritative for payment state; user redirect alone does not prove payment success.
- A cancellation after capture must trigger an explicit refund workflow, not simply change the order label.
- Partial shipment/refund support must either be implemented explicitly or excluded by launch policy. Do not imply full support if only whole-order actions are built.

7.4 Coupons

- Coupon validity includes start/end, status, minimum cart value, eligible products/categories, per-customer/global limits and allowed discount types.
- The backend validates and reserves/consumes use consistently with order confirmation and rollback behavior.
- UI errors must be informative without exposing internal rule implementation or sensitive customer information.

7.5 Returns and cancellations

- Product owner must specify cancellation cutoff, shipped-order cancellation handling, return window, final-sale exclusions, evidence requirements, who approves a return, restocking rules, refund timing, partial-return support and notification policy before S9 implementation is considered done.

8. UX, responsive behavior and accessibility

- Mobile-first responsive behavior for storefront, cart, checkout and admin's core flows.
- Define consistent type scale, spacing, input heights, button variants, border/focus states, error states, loading skeletons, empty states and confirmation dialogs.
- All form fields have programmatic labels; inline errors are associated with controls; keyboard focus remains visible and dialogs manage focus correctly.
- Core journeys support keyboard-only use; color is not the sole indicator of status; text contrast and touch target sizes are checked.
- Provide loading, success, no-results, empty-cart, out-of-stock, expired-coupon, session-expired, payment-failed and network-error states.
- Target WCAG 2.2 AA-oriented verification; do not treat automated accessibility scan alone as sign-off.

9. SEO and content requirements

- Public category and product routes should be server-rendered or statically rendered where appropriate with canonical URLs and unique metadata.
- Generate XML sitemap for eligible public URLs; use robots rules/noindex for account, admin, cart, checkout, search parameters where appropriate, and draft/unpublished product pages.
- Prevent duplicate indexing of pagination/filter combinations using a documented canonical/indexing strategy.
- Implement redirects when product/category slugs change and avoid broken image or product URLs.
- Add suitable structured data only when fields are accurate; never fabricate review/rating values.
- Product copy, category copy, alt text, policy pages, logo/favicons and launch content are dependencies owned by Product/Content.

10. Technical architecture and standards

10.1 Proposed logical components

- **Web storefront/admin:** Next.js + TypeScript; Redux Toolkit for shared client state where beneficial; server state should use a deliberate fetching/cache strategy rather than duplicating every API response in Redux.
- **API/service layer:** Node.js + Express + TypeScript; Zod schemas at boundaries; consistent API envelope, error codes, request IDs, logging and authorization.
- **Suggested bounded contexts:** Identity/Profile, Catalog/Search, Cart/Pricing/Promotions, Inventory, Checkout/Orders, Payments adapter, Notifications, Admin/Operations. Start with clear logical module boundaries and split into independently deployed microservices only where ownership, scaling or reliability benefits justify the operational overhead.
- **Primary database:** PostgreSQL on managed AWS database service; Prisma ORM and reviewed migrations. Define data ownership per service; avoid cross-service writes to another service's tables.
- **Cache / session / rate-limit support:** Redis/ElastiCache where justified for cache, ephemeral state or rate limits; it is not the source of truth for orders/payments.
- **Assets:** S3 for product media and generated documents; CloudFront for controlled CDN delivery.
- **Async work:** SQS/EventBridge or equivalent for email, webhook follow-up, reconciliation and non-blocking jobs; bounded retries and dead-letter queue.
- **Hosting:** AWS ECS/Fargate or an approved managed container platform; ALB/API routing; IaC; separate dev/staging/production; Secrets Manager/Parameter Store; CloudWatch metrics/logs/alarms. Final service choices require an architecture decision record.
- **CI/CD:** Git-based PR checks, unit/integration/contract tests, image/dependency scanning, environment promotion, migration checks, smoke tests and rollback path.

10.2 Service boundaries and reliability

- Each service/module owns its business rules and data writes. APIs/events are versioned and documented.
- Avoid distributed transactions across PostgreSQL databases. Use explicit state machines, idempotent consumers, outbox/inbox or reconciliation patterns where needed.
- Set request timeouts; retry only idempotent operations or operations with idempotency keys. Use circuit-breaking/backoff where appropriate.
- Payment webhook handlers verify authenticity, persist the event, acknowledge promptly, and process safely even if provider retries or events arrive out of order.
- Use correlation IDs across HTTP, queue messages, logs and traces.

10.3 API conventions

- Version public/internal APIs (for example /api/v1/...) and document them with OpenAPI.
- Standard response shape should include data/meta on success and stable error code/message/field details on failure; never leak stack traces.
- Use pagination and deterministic sorting for list APIs; enforce maximum page size.
- Use Zod for body/query/path validation and normalize unknown errors in centralized middleware.
- Enforce authentication and resource ownership on every protected endpoint. CORS is explicit, not wildcard for credentialed origins.
- Apply rate limits and abuse controls to login, reset, search and other expensive endpoints.

11. Core data model (logical, not final physical schema)

Entity	Important fields / relationships	Notes
User	id, email, password hash/provider id, verification status, status, timestamps	Never store plaintext password; avoid using email as immutable primary key
Role / Permission	role id, permission key, assignment	Default deny; changes are audited
Address	user id, recipient, address lines, locality, region, postal code, country, phone if needed	Order stores a snapshot to preserve history
Category	id, parent id, name, slug, status, sort order	Prevent hierarchy cycles and slug collision
Product	id, category refs, title, slug, description, status, media, SEO fields	Draft/archived products not customer-purchasable
ProductVariant / SKU	id, product id, SKU, attributes, price, currency, status	Unique SKU; variant-specific availability/pricing supported
Inventory	SKU id, on-hand/reserved/available quantities, version	Update transactionally; audit movements
InventoryMovement	SKU, delta, reason, actor, before/after, order/ref	Immutable audit record
Cart / CartItem	cart owner or guest key, SKU, quantity, timestamps	Server recalculates pricing and availability
Wishlist / WishlistItem	user id, product/variant id, created at	Private to owner
Coupon / Promotion	code, type, value, dates, eligibility, limits, usage	Enforce use limits atomically
Order / OrderItem	order number, user/guest reference, money breakdown, snapshots, status	Immutable financial snapshot; no client-controlled final totals
Payment / PaymentEvent	order id, provider ref, amount/currency, status, idempotency key; event id	Do not store raw card data; redact provider payloads before logs
Refund / ReturnRequest	order/item, amount, reason, status, approver, provider ref	Prevent over-refund and duplicate refund
Shipment	order id, carrier, tracking number/url, status, timestamps	Use provider/manual tracking model selected for launch
Review	user, product, rating, content, moderation state, order eligibility	Subject to policy and moderation

Notification / Job	type, recipient reference, template, status, attempts, provider ref	Do not log sensitive content or secrets
AuditLog	actor, action, entity type/id, timestamp, correlation id, safe metadata	Append-only access policy; retention defined

Data decisions required: UUID/ID strategy, deletion/anonymization behavior, retention rules, currency representation, tax/invoice fields, PII access and encryption, guest checkout retention, order-number format, and whether partial refunds/shipments are launch requirements.

12. API surface (indicative; exact contracts belong in OpenAPI)

Domain	Representative endpoints
Identity	POST /api/v1/auth/register, /login, /logout, /refresh, /verify-email, /forgot-password, /reset-password
Profile	GET/PATCH /api/v1/me, GET/POST/PATCH/DELETE /api/v1/me/addresses
Catalog	GET /api/v1/categories, GET /api/v1/products, GET /api/v1/products/:slug, admin product/category CRUD
Search	GET /api/v1/search?q=&category=&minPrice=&maxPrice=&sort=&page=
Cart	GET /api/v1/cart, POST /api/v1/cart/items, PATCH/DELETE /api/v1/cart/items/:id
Wishlist	GET /api/v1/wishlist, POST /api/v1/wishlist/items, DELETE /api/v1/wishlist/items/:id
Promotions	POST /api/v1/cart/coupon, DELETE /api/v1/cart/coupon; admin coupon CRUD
Checkout	POST /api/v1/checkout/preview, POST /api/v1/checkout/submit with idempotency key
Payments	POST /api/v1/payments/session, POST /api/v1/webhooks/payments/:provider
Orders	GET /api/v1/me/orders, GET /api/v1/me/orders/:orderNumber; role-gated admin search and updates
Returns/refunds	Customer return/cancel request; admin approval/processing; provider refund adapter
Media	Admin-authorized pre-signed upload request and media association endpoints
Health/ops	/health, /ready, internal metrics and queue/reconciliation visibility with access control

All paths are examples for ticket planning, not a finalized public contract. Define request/response schemas, status codes, authorization, idempotency and error cases in OpenAPI per endpoint.

13. Security, privacy and compliance

- Apply OWASP ASVS/Top 10-informed review: access control, input validation, output encoding, injection prevention, XSS/CSRF controls, SSRF protections where URLs are fetched, secure headers, restrictive CORS, rate limiting and dependency scanning.
- Passwords use a modern password-hashing algorithm and parameters selected by security review; secrets are never committed or written to logs.
- Use HTTPS end-to-end; managed encryption at rest for database, storage and backups; least-privilege IAM; secret rotation procedures.
- Do not store PAN/CVV or raw card details. Use a payment provider's hosted/tokenized flow and confirm applicable PCI responsibility with the provider/acquirer; do not claim certification without evidence.
- Verify webhook signatures, use timestamps/replay controls where supported, and avoid trusting browser redirects as payment confirmation.
- Minimize PII access; mask sensitive admin views; audit data exports and refund/permission changes.
- Rate-limit credential and high-risk endpoints; add abuse detection and alerts.
- Define privacy notice, cookie/analytics consent where applicable, deletion/anonymization requests, retention schedule, and incident-response process with legal/business owners.
- Perform dependency/container/IaC scanning and remediate critical/high findings before launch or record formal risk acceptance.

14. Non-functional requirements

Area	Requirement / launch gate
Performance	Establish representative traffic profile; target p75 LCP <= 2.5s on key public pages and p95 <= 500ms for key read APIs under agreed load, excluding third-party latency. Checkout/payment latency is measured separately.
Availability	Initial target 99.5% monthly; define exclusions, alert thresholds and ownership. Do not promise this without monitoring and operational support.
Scalability	Stateless web/API instances where practical; managed DB connection pooling; pagination; query/index review; CDN for static/media assets; queue for non-critical async work.
Reliability	Timeouts, bounded retries, idempotency, graceful degraded states, webhook replay safety and reconciliation runbook.
Accessibility	WCAG 2.2 AA-oriented manual and automated review for core journeys; keyboard/focus/form errors tested.
Browser/device	Define supported current browser versions and mobile/desktop breakpoints in Sprint 1; verify Chromium, Firefox, Safari and representative mobile devices.
SEO	Renderable public pages, canonical/metadata, sitemap/robots and redirect checks; private pages noindex.
Maintainability	TypeScript strictness, linting, code review, tests, OpenAPI docs, ADRs and repeatable local setup.
Observability	Structured logs, metrics, request correlation, key business metrics, alerts, dashboards and runbooks.
Recovery	Automated database backups; perform restore test; set business-

	approved RPO/RTO before launch.
Data integrity	Transactional stock/order writes, immutable order snapshots, payment reconciliation and migration tests.
Compatibility	API contracts/versioning documented; migrations are backward-compatible where deployment order requires it.

15. Testing and quality strategy

- **Unit tests:** pricing, coupon eligibility, inventory reservations, order/payment transitions, authorization policies, Zod schemas and refund limits.
- **Integration tests:** service/database boundaries, transaction behavior, migrations, queue handlers, object storage, email provider adapter and payment provider sandbox.
- **Contract tests:** OpenAPI request/response behavior, event schemas, provider webhook fixtures and inter-service compatibility.
- **E2E tests:** sign-up/login, browse/search/filter, product variant, cart/merge, coupon, checkout success/decline/cancel, order history, admin stock/order update, cancellation/return/refund.
- **Security tests:** privilege escalation, IDOR, session rotation/revocation, rate limits, XSS/CSRF/injection, webhook signature/replay, dependency/image scan, secrets scan.
- **Performance tests:** listing/detail/cart/checkout and database/query hotspots at agreed concurrency; record baseline and remediation.
- **Accessibility tests:** automated scan plus keyboard/screen-reader spot checks of core journeys and error states.
- **UAT:** product, operations, support and finance/owner walk through realistic scripts with evidence and sign-off.
- **Regression policy:** P0 journey suite runs on every release candidate; critical flows cannot be waived without named risk acceptance.

16. Analytics and observability

16.1 Product events

Define a versioned event dictionary for: product_viewed, search_submitted, filter_applied, add_to_cart, remove_from_cart, coupon_applied, checkout_started, payment_succeeded, payment_failed, order_placed, order_shipped, cancellation_requested, refund_completed, review_submitted.

Include only necessary event properties such as event ID, timestamp, product/SKU identifiers, currency, safe amount, anonymous/session ID and consent state. Never send credentials, full address, payment instrument details or unnecessary free text.

16.2 Operational metrics

- HTTP request rate, p50/p95/p99 latency and 4xx/5xx rates by service.
- Database saturation, connection pool use, slow queries, replication/backup health where relevant.
- Queue depth, oldest message age, retries and dead-letter count.
- Checkout start/submit failures, payment success/failure rates, webhook lag, stuck pending orders and reconciliation mismatches.
- Inventory reservation expirations, negative-stock invariant violations and refund failures.

- Deployment version, health checks, uptime and alert acknowledgement.

17. Rollout and sprint plan

The separate sprint workbook contains the full roadmap and ticket-level backlog. High-level rollout:

Sprint	Dates	Rollout / milestone
S1	26 Oct–6 Nov 2026	Discovery, agreed scope, architecture, environment and UX foundation
S2	9–20 Nov 2026	App/API shell, identity, sessions, RBAC and reusable UI
S3	23 Nov–4 Dec 2026	Catalog, listing, search/filter/sort and admin catalog v1
S4	7–18 Dec 2026	Product detail/variants, media and inventory foundation
S5	21 Dec 2026–1 Jan 2027	Cart, wishlist, pricing and coupons; reduced holiday capacity
S6	4–15 Jan 2027	Checkout, address, shipping/tax approach, stock reservation and payment sandbox
S7	18–29 Jan 2027	Webhooks, order/payment state, customer order history and notifications
S8	1–12 Feb 2027	Admin/operations, fulfilment, stock/coupon management and audit
S9	15–26 Feb 2027	Cancellation/returns/refunds, reviews, notifications, policy pages and E2E journeys
S10	1–12 Mar 2027	Regression, security, load, accessibility, SEO, monitoring, backup and release candidate 1
S11	15–26 Mar 2027	UAT, remediation, cutover rehearsal, live provider readiness and go/no-go
Release	29–31 Mar 2027	Production deployment and monitored launch

Capacity caution: This scope is ambitious for one developer. It assumes a cross-functional team with frontend, backend, QA, product/design, cloud/DevOps and operations coverage, or a formal scope reduction. Story points are relative planning inputs, not commitments; re-estimate after team capacity is known. Do not compress security, payment correctness or release testing to preserve optional features.

18. Dependencies, risks and mitigations

Risk/dependency	Impact	Mitigation / owner
Payment provider or production account selected late	Checkout, webhooks and launch verification blocked	Select provider and complete onboarding in S1; use sandbox early; owner: PM/Finance/Backend
Launch country/currency/tax/invoice rules unclear	Price/tax/order correctness and legal readiness at risk	Approve launch market, currency, tax model and invoice fields in S1; owner:

		Product/Finance/Legal
Shipping and return policies undefined	Checkout and post-purchase flows cannot be finalized	Approve shipping regions/rules, delivery estimate, cutoff and refund policy early; owner: Product/Ops
Microservices over-fragmentation	Operational complexity, distributed failures and delayed delivery	Start with bounded contexts; split into deployed services only where there is a clear benefit; owner: Architect
No product/content data or image rights	Catalog/UAT launch incomplete	Assign catalog/content owner; define import template and content freeze date; owner: Product/Content
Holiday sprint capacity loss	Cart/pricing schedule slips	Plan reduced capacity for S5; front-load decisions; protect P0 scope; owner: PM
Payment webhook duplication/out-of-order events	Duplicate orders, incorrect paid status or refunds	Idempotency, state machine, provider event ledger, replay tests and reconciliation; owner: Backend/QA
Inventory race conditions	Overselling or stranded reservations	Transactional reservations, expiry, concurrency tests and alerting; owner: Backend
Scope creep / marketplace expectation	March launch threatened	Keep single-store launch boundary; use change control and P1/P2 triage; owner: Product owner
Team capacity/skills unknown	Plan dates may be unrealistic	Confirm staffing and available days in S1; use velocity after first two sprints to reforecast; owner: Engineering Lead/PM
Weak operational readiness	Extended outage or unhandled payment failures	Runbooks, backup restore, alerting, rollback and support training by S11; owner: Ops/DevOps

19. Open decisions and approvals

Resolve the following by the end of S1 unless noted:

15. Launch country/region, supported currencies, languages and address formats.
16. Payment provider, hosted/tokenized integration mode, production account, webhook and refund capabilities.
17. Shipping providers/methods, eligible locations, cost/thresholds, estimated delivery and tracking policy.
18. Tax rules/provider, invoice/legal fields, price display tax-included vs tax-exclusive.
19. Guest checkout policy and whether account creation is mandatory.
20. Cancellation cut-off, return window, exclusions, approval flow, restocking and partial refund/shipment requirements (must be approved before S9).
21. Coupon stacking, usage limits and discount precedence.
22. Review eligibility and moderation ownership; reviews can move to post-launch if policy/ownership is not ready.
23. Launch catalog size, content/image owner, opening inventory source and data migration format.

10. Supported browsers/devices, accessibility target and analytics/consent obligations.
11. Target traffic/concurrency, monthly AWS budget, uptime objective, RPO/RTO and incident coverage.
12. Final team capacity, named owners, QA/ops coverage and go-live approvers.

20. Definition of Ready and Definition of Done

Definition of Ready

- User/problem and business outcome are clear.
- In-scope behavior and non-goals are explicit.
- Acceptance criteria cover happy path, validation, authorization, loading/empty/error states and edge cases.
- Dependencies, design, API/data contract, test approach, telemetry and security/privacy impact are identified.
- Product decisions and policy dependencies are not hidden as assumptions.
- Story is sized by the team and fits within a sprint with test/QA effort included.

Definition of Done

- Code is reviewed, type-safe, linted and merged through protected CI.
- Unit/integration/contract tests are added or updated; P0 flows have automated coverage.
- API/OpenAPI and user/admin documentation are updated.
- Authorization, validation, error handling and audit/telemetry are implemented where applicable.
- Accessibility/responsive behavior, empty/loading/error states and cross-browser impact are checked.
- No secret/PII leakage; security scans and migration checks pass.
- Feature is deployed to the target environment, verified by QA, and accepted against criteria.
- Any known issue has severity, owner, workaround and explicit acceptance or release-blocking status.

21. Ticket template for the PM

Every dev ticket should contain:

- **ID / Epic / Priority / Sprint / Owner role**
- **User story:** As a [persona], I want [capability], so that [outcome].
- **Context and scope:** What is included and explicitly excluded.
- **Acceptance criteria:** Numbered, observable outcomes including error/edge cases.
- **API/data contract:** Endpoints/events, Zod schema, authorization, validation, state transitions and migrations.
- **UI states:** loading, success, empty, validation error, permission denied, network failure, retry.
- **Security/privacy:** PII handling, role checks, abuse controls, audit requirements, secrets.
- **Telemetry:** Events, logs, metrics, alert/runbook if relevant.
- **Dependencies:** Ticket IDs and external approvals.
- **Test notes:** unit/integration/E2E/UAT evidence.
- **Definition of done and release flag:** feature flag, migration/release notes, rollback considerations.

22. Launch acceptance checklist

- ☐ All P0 requirements implemented and accepted; P1 features are either complete or formally deferred.
- ☐ Guest/customer purchase journey and failure/retry paths pass in the production-like environment.
- ☐ Payment success, decline, cancel, duplicate webhook, refund and reconciliation have evidence.
- ☐ No unresolved Sev-1/Sev-2 defect and no unresolved critical/high security issue without formal acceptance.
- ☐ Core keyboard/accessibility review, SEO crawl and target performance test pass.
- ☐ Product/catalog/opening inventory and policy/legal content are approved and loaded.
- ☐ Monitoring, alert routing, dashboards, support escalation, backups and restore evidence are ready.
- ☐ Production deployment, migration, smoke test and rollback have been rehearsed.

- ☐ Payment provider production configuration and webhook secrets have been verified.
- ☐ Operations/support staff know order lookup, inventory correction, cancellation/refund and incident procedures.
- ☐ Product owner, engineering lead, QA, operations and business approver record go/no-go.
- ☐ Launch-day monitoring window and decision-maker are assigned for 29–31 March 2027.

Document ownership: Product Manager (scope and policy), Engineering Lead (architecture and technical standards), QA Lead (verification and evidence), Operations/Finance (fulfilment, payment and refund readiness). Revisit scope and timeline after S2 velocity and at every sprint review.